# ProEm: Formal Verification with Open-Source Tools

Yosys, SymbiYosys, yices and ABC, from first principles to the proofs in `formal/`

Paul Vincent Bezzina

29 September 2026

**Abstract**

This guide explains formal verification for someone who knows simulation but has not used a model checker. It covers the ideas (assertions, assumptions, covers, bounded and unbounded proofs), the open-source tools and how to install them on Windows, how to write a proof harness, and how to read the results. Every example comes from the `formal/` folder of the ProEm repository, where four modules (the host FIFO, the frame buffer, the UART transmitter and the branch unit) are proven with no depth limit. Each proof takes about a second.

## Contents

**1 What formal verification is** **3**

**2 The tools** **3**
2.1 Installing on Windows (WSL) . . . 3

**3 The four building blocks** **4**
3.1 `assert`: what must always be true . . . 4
3.2 `assume`: what the environment promises . . . 4
3.3 `cover`: proof that something can happen . . . 4
3.4 Free values: inputs, `anyconst`, `anyseq` . . . 4

**4 How the tool searches: bounded and unbounded proofs** **4**

**5 The harness pattern** **5**
5.1 Why a separate harness . . . 5
5.2 A reference model . . . 5
5.3 Checking data with one tracked value . . . 5
5.4 The `.sby` file . . . 6

**6 Running proofs and reading the results** **7**
6.1 When a proof fails . . . 7
6.2 Checking that a proof has teeth . . . 7

**7 The four ProEm proofs** **7**
7.1 Host FIFO (`host_fifo_formal.sv`) . . . 8
7.2 Frame buffer (`frame_buffer_formal.sv`) . . . 8
7.3 UART transmitter (`uart_tx_formal.sv`) . . . 8
7.4 Branch unit (`branch_unit_formal.sv`) . . . 8

**8 Pitfalls** **8**

**9 Next steps** **9**

**10 Glossary** **9**

# 1 What formal verification is

A simulation test drives chosen inputs and checks the outputs. However many tests you write, you only learn about the input sequences you tried. Your 1,800 random programs are a lot, but a FIFO bug that needs "write, flush, read and write in the same cycle, while full" may never be generated.

A formal tool works the other way round. You state rules that must always be true (**assertions**), and the tool searches *every* possible input sequence, mathematically, for one that breaks a rule. Either it proves that none exists, or it hands you the shortest input sequence that breaks it, as a waveform (a **counterexample**).

| | Simulation | Formal |
|---|---|---|
| Inputs | The ones your test chooses | All possible inputs, every cycle |
| Answer | "No failure seen" | "Proven" or "here is a failing trace" |
| Best for | Whole-chip behaviour, long protocols | Control logic: FIFOs, pointers, state machines, arbitration |
| Weak at | Rare corner cases | Very large designs, long counters, arithmetic |

The two complement each other. ProEm uses simulation for the protocols and the whole chip, and formal proofs for the small blocks whose corner cases are hardest to reach by testing.

# 2 The tools

| Tool | Role |
|---|---|
| Yosys | Reads your SystemVerilog and the formal harness and turns them into a mathematical model. You already use it for synthesis. |
| SymbiYosys (SBY) | The driver. It reads a small `.sby` file, runs Yosys, calls the solver, and reports PASS or FAIL. |
| yices, boolector, bitwuzla, z3 | **SMT solvers**: programs that answer "is there any assignment of values that makes this formula true?". SBY uses them through `yosys-smtbmc`. yices is usually the fastest for hardware. |
| ABC (`pdr`) | A model checker that proves properties with no depth limit by finding an invariant automatically (Section 4). |
| GTKWave or Surfer | Viewers for counterexample and cover waveforms (`.vcd`). |

The simplest way to get all of them, at matching versions, is the **OSS CAD Suite**, a single download from YosysHQ.

## 2.1 Installing on Windows (WSL)

1. Open the Ubuntu (WSL) terminal.
2. Go to `https://github.com/YosysHQ/oss-cad-suite-build/releases`, and copy the link to the newest `oss-cad-suite-linux-x64-YYYYMMDD.tgz`.
3. Download, extract and activate it:

```
cd ~
wget <the link you copied>
tar -xzf oss-cad-suite-linux-x64-*.tgz
source ~/oss-cad-suite/environment    # run this in every new terminal
sby --help                             # check it works
```

To make the last `source` line automatic, add it to the end of `~/.bashrc`. The `formal` GitHub workflow installs the same suite with the `YosysHQ/setup-oss-cad-suite` action.

# 3 The four building blocks

A formal harness is ordinary SystemVerilog plus four statements.

## 3.1 `assert`: what must always be true

```
assert (full == (count == DEPTH));
```

The tool tries to make this false. If it cannot, the property is proven.

## 3.2 `assume`: what the environment promises

```
initial assume (!rst_n);        // the design starts in reset
always_comb assume (ratio != 0); // the divider is never programmed to 0
```

Assumptions restrict the inputs the tool may choose. They are powerful and dangerous: an assumption that is too strong can rule out the very case that would fail, and the proof then passes without meaning anything. Keep them few, and only for things the rest of the chip really guarantees.

## 3.3 `cover`: proof that something can happen

```
cover (full);   // show me a trace where the FIFO fills up
```

A cover asks the tool to *find* a trace where the condition is true, and writes it as a waveform. Covers catch **vacuous** proofs: if the FIFO can never become full under your assumptions, every assertion about "when full" passes trivially, and the cover exposes that.

## 3.4 Free values: inputs, `anyconst`, `anyseq`

Every input port of the harness is free: the tool may give it any value in any cycle. For a value that is chosen once and then held, mark a signal `(* anyconst *)`:

```
(* anyconst *) logic [3:0] ratio;   // any divider from 0 to 15, fixed for the run
```

That single line makes the UART proof cover every clocks-per-bit setting at once.

# 4 How the tool searches: bounded and unbounded proofs

**Bounded model checking** (`mode bmc`)
: Checks every input sequence up to a fixed length, say 30 cycles from reset. It is simple and finds real bugs quickly, but it says nothing about cycle 31. The solver's work also grows quickly with depth: on the frame buffer, 20 cycles already took minutes.

**k-induction** (`mode prove` with `smtbmc`)
: Proves "if the assertions held for $k$ cycles in a row, they hold in the next one too" from *any* state, not only reachable ones. It often fails on harmless designs, because the tool starts from impossible states. For example, a harness counter that disagrees with the FIFO's internal pointers can never happen, but induction does not know that. Fixing this needs extra assertions about the design's internal signals, which open-source Yosys cannot reach from a separate harness.

**PDR, also called IC3 (`mode prove` with `abc pdr`)**
: Builds the missing invariant automatically, by learning which states are unreachable. For control logic like ProEm's buffers and state machines it proves properties with **no depth limit**, usually in a second. This is what ProEm's proofs use.

Covers always run as a bounded search, since they only need one example.

# 5 The harness pattern

Each ProEm proof has two files in `formal/`: a harness (`<module>_formal.sv`) and a job description (`<module>.sby`). The RTL in `src/` is not modified.

## 5.1 Why a separate harness

- Your RTL stays exactly as it is synthesised; nothing is added under `ifdef`s.
- The harness only uses the module's ports, so the properties describe the behaviour the rest of the chip relies on, not how the module is built. If you rewrite the module, the same proof still applies.
- The harness can shrink parameters (a 4-entry buffer instead of 16) to keep the proof fast. The logic is the same for any size.

## 5.2 A reference model

The harness keeps a small model of what the module *should* be doing, built only from its inputs and outputs, and asserts that the module agrees with it. For the host FIFO the model is a single counter:

```
assign wrFire = wr_en && !full && !flush;
assign rdFire = rd_en && !empty && !flush;

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        count <= '0;
    end else if (flush) begin
        count <= '0;
    end else begin
        count <= count + wrFire - rdFire;
    end
end

always_comb begin
    if (started && rst_n) begin
        assert (count <= DEPTH);
        assert (empty == (count == 0));
        assert (full == (count == DEPTH));
    end
end
```

`started` is a flag that is 0 in the very first cycle, so no assertion is checked before reset has been applied.

## 5.3 Checking data with one tracked value

Counting proves the flags, but not that data comes out unchanged and in order. Checking every entry would need a model as large as the FIFO. The standard trick is to follow *one* value, chosen

by the tool:

- A free input `pick` marks the write the tool wants to follow. The harness records its value and how many entries are ahead of it.
- Every read moves it one place closer to the front.
- When it reaches the front, the FIFO's output must equal the recorded value.

```
if (!tracking && wrFire && pick) begin
    tracking <= 1'b1;
    trackedValue <= din;
    trackedPos <= count - rdFire;
end
...
if (tracking && trackedPos == 0) begin
    assert (dout == trackedValue);
end
```

Because the tool may pick *any* write, with *any* value, at *any* time, proving it for the one tracked value proves it for all of them.

### 5.4 The `.sby` file

```
[tasks]
prove
cover

[options]
prove: mode prove
cover: mode cover
cover: depth 30

[engines]
prove: abc pdr
cover: smtbmc yices

[script]
read -formal host_fifo.sv host_fifo_formal.sv
prep -top host_fifo_formal
memory_map

[files]
../src/primitives/host_fifo.sv
host_fifo_formal.sv
```

`[tasks]`
: Two jobs from one file: `prove` (the assertions) and `cover` (the covers). A line starting `prove:` only applies to that task.

`[options]`
: The mode of each task, and the search depth for the cover.

`[engines]`
: PDR for the proof, and yices through `smtbmc` for the covers.

`[script]`
: The Yosys commands. `read -formal` enables `assert`, `assume` and `cover`. `prep -top` picks the

harness as the top module. `memory_map` turns the FIFO's storage array into ordinary registers, which the solvers handle much faster.

`[files]`
The files SBY copies into its work folder.

SBY also converts asynchronous resets for you, so `always_ff @(posedge clk or negedge rst_n)` needs no special handling.

# 6 Running proofs and reading the results

```
cd ProEm
bash scripts/formal.sh          # all proofs
cd formal && sby -f host_fifo.sby   # one proof, both tasks
sby -f host_fifo.sby prove          # one task only
```

`scripts/formal.sh` prints one line per module:

```
PASS  branch_unit (prove and cover)
PASS  frame_buffer (prove and cover)
PASS  host_fifo (prove and cover)
PASS  uart_tx (prove and cover)
```

Each task writes a folder such as `formal/host_fifo_prove/`. The last lines of its `logfile.txt` say `DONE (PASS, rc=0)` or `DONE (FAIL, rc=2)`.

## 6.1 When a proof fails

The log names the failing assertion by file and line, and the folder holds a counterexample:

```
Assert failed in frame_buffer_formal: frame_buffer_formal.sv:113 ...
counterexample trace: frame_buffer_prove/engine_0/trace.vcd
```

Open `trace.vcd` in GTKWave. It is the shortest input sequence, from reset, that breaks the assertion. Read it like a failing simulation: find the cycle where the assertion fails, then walk back to see which input caused it. Then decide whether the RTL is wrong, or the property or an assumption is.

## 6.2 Checking that a proof has teeth

A proof that cannot fail is worthless, so plant a bug on purpose and confirm the proof catches it. Two examples from ProEm, done on a copy of the RTL:

| Planted bug | Result |
|---|---|
| Frame buffer: rollback restores the write pointer to `cm_ptr + 1` instead of `cm_ptr` | FAIL at once, with a trace |
| Branch unit: rule 1 given priority over rule 0 | FAIL at once, with a trace |

Covers do the same job in the other direction: each cover must be *reached*, or the property it guards may be vacuous.

# 7 The four ProEm proofs

All are unbounded (PDR) and each takes about a second.

### 7.1 Host FIFO (`host_fifo_formal.sv`)

Occupancy never exceeds the depth; `full` and `empty` match the count exactly; `flush` empties it; any value written comes out unchanged and in order. Covers: the FIFO fills; a tracked value travels the whole depth.

### 7.2 Frame buffer (`frame_buffer_formal.sv`)

The model keeps two counts: entries written and not yet read (`total`), and entries committed and not yet read (`committed`). It proves:

- `committed` $\leq$ `total` $\leq$ depth, and `full`, `almost_full` and `empty` are exact (`empty` is based on committed entries only, so the reader never sees a half-built frame);
- rollback discards exactly the uncommitted entries (a tracked value that was not yet committed is dropped; a committed one survives);
- `commit_all` includes a write in the same cycle;
- a tracked value appears on `dout` the cycle after it is read (the output is registered).

### 7.3 UART transmitter (`uart_tx_formal.sv`)

With `ratio` free (`anyconst`, 1 to 15), the model counts bits and cycles, and the proof shows every frame is a start bit, eight data bits LSB first and a stop bit, each held for exactly `ratio` clocks; the line is high whenever idle; and `tx_ready` is high exactly when idle. Cover: a full frame of 0xA5 at 3 clocks per bit.

### 7.4 Branch unit (`branch_unit_formal.sv`)

The harness models the two BRANCH rules from the ISA, loaded through the same configuration writes: compare the chosen block's byte under the mask with the value; take the hit or miss action; for a read-back rule, fire on a mismatch; rule 0 wins over rule 1; each rule acts at most once per frame, until the next frame starts or the rule is rewritten. It proves that `set_mode` and `new_mode` give the chosen action in the same cycle and only inside a frame; that `hit` and `branch_abort` follow one cycle later (they are registered) and are suppressed by a frame reset; that `stall` is high for exactly the decision cycle and the abort cycle; and that `line_watch` is set exactly when a read-back rule is enabled. Covers: a switch to read; an abort after a hit; both rules firing together.

## 8 Pitfalls

**Too-strong assumptions**
An assumption that removes real inputs makes a proof meaningless. Every assumption should describe something the surrounding logic truly guarantees. Covers help catch this.

**Checking during reset**
Guard assertions with `started && rst_n`, or the tool will "fail" on the first cycle before reset has taken effect.

**Memories**
Arrays make the solvers slow or fail; `memory_map` in the script fixes this for small arrays.

**Large parameters**
Proof time grows quickly with widths and depths. Shrink parameters in the harness when the logic does not depend on them.

**Long counters**
A 16-bit baud counter has 65,536 states per bit. Constrain the value range, or shrink the counter width in the harness, as the UART proof does with a 4-bit ratio.

**Solver differences**
If one engine is slow, try another: `abc pdr` for unbounded proofs, `smtbmc yices` for bounded proofs and covers.

## 9 Next steps

1. **UART receiver.** Prove that a byte sent at the measured rate is received correctly. Auto-baud makes this harder: start by assuming the rate is already locked (a free `anyconst` ratio), then prove the measurement separately.
2. **Frame chain counters.** START, the headers, MAIN and END each count bits; prove each counts exactly the programmed number and hands over to the next block.
3. **Line codec pieces.** For example, prove that bit stuffing followed by unstuffing gives back the original bits, by connecting the encoder to the decoder in a harness.
4. **A future CRC unit.** Prove it against a simple reference model (an obviously-correct bit-serial CRC) before it goes into the chip.

## 10 Glossary

**Assertion**
A condition that must hold in every reachable state.

**Assumption**
A constraint on inputs; the tool only considers inputs that satisfy it.

**BMC**
Bounded model checking: all input sequences up to a fixed depth.

**Counterexample**
A trace, from reset, that breaks an assertion.

**Cover**
A condition the tool must show is reachable.

**Harness**
A wrapper module that instantiates the design and adds the model and properties.

**Induction**
Proving that a property, if true for $k$ cycles, stays true in the next.

**Invariant**
A property that holds in every reachable state.

**PDR / IC3**
An algorithm that finds invariants automatically and proves properties without a depth limit.

**Reference model**
A simple description of the intended behaviour, compared against the design.

**SBY**
SymbiYosys, the driver that runs Yosys and the solvers from a `.sby` file.

**SMT solver**
A program that decides whether a logical formula over bit-vectors can be satisfied.

**Vacuous proof**
A proof that passes only because the situation it talks about can never occur.